

Cascade ST Baseline experiment using Whisper along with NLLB and Europarl-ST (Spanish to English)

In this notebook, we are going to learn how to use Whisper and Meta's large pre-trained models [NLLB](#) in a cascade ST approach on the Europarl-ST using the English-Spanish translation pair.

First, we import automatic transcriptions, references and translations (and their clean counterparts) generated by the ASR baseline experiment using Whisper

```
In [ ]: from datasets import load_dataset

raw_datasets = load_dataset("csv", data_files="L4.1_ASR_Whisper_Baseline_Europarl-ST.csv")

print(raw_datasets)
```

Show the first sample for each dataset feature

```
In [ ]: print(raw_datasets["train"][:1]["hypothesis"])
print(raw_datasets["train"][:1]["reference"])
print(raw_datasets["train"][:1]["translation"])
print(raw_datasets["train"][:1]["hypothesis_clean"])
print(raw_datasets["train"][:1]["reference_clean"])
print(raw_datasets["train"][:1]["translation_clean"])
```

Now we load the pre-trained tokenizer for the NLLB model and apply it to the Spanish-English pair:

```
In [3]: max_tok_length = 275

from transformers import AutoTokenizer

checkpoint = "facebook/nllb-200-distilled-600M"
# from flores200_codes import flores_codes
src_code = "spa_Latn"
tgt_code = "eng_Latn"
tokenizer = AutoTokenizer.from_pretrained(
    checkpoint,
    padding=True,
    pad_to_multiple_of=8,
    src_lang=src_code,
    tgt_lang=tgt_code,
    truncation=False,
    max_length=max_tok_length,
)
```

We will use the `Dataset.map()` function. This allows us some extra flexibility, if we need more preprocessing done than just tokenization. The `map()` method works by applying a function on each element of the dataset.

In our case, each sample pair is going to be preprocessed according to the training needs of the model that is to be used. In this case we tokenize speech transcriptions and translations to perform ST cascade experiments

```
In [4]: def preprocess_function(sample):
        model_inputs = tokenizer(
            sample["hypothesis"],
            text_target = sample["translation"],
        )
        return model_inputs
```

Now, we can apply the `preprocess_function` to the raw datasets

```
In [ ]: tokenized_datasets = raw_datasets.map(preprocess_function, batched=True)
```

bitsandbytes is a quantization library with a Transformers integration. With this integration, you can quantize a model to 8 or 4-bits and enable many other options by configuring the BitsAndBytesConfig class. For example, you can:

- set load_in_4bit=True to quantize the model to 4-bits when you load it
- set bnb_4bit_quant_type="nf4" to use a special 4-bit data type for weights initialized from a normal distribution
- set bnb_4bit_use_double_quant=True to use a nested quantization scheme to quantize the already quantized weights
- set bnb_4bit_compute_dtype=torch.bfloat16 to use bfloat16 for faster computation

```
In [6]: import torch
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
)
```

Pass the quantization_config to the from_pretrained method.

```
In [ ]: from transformers import AutoModelForSeq2SeqLM

model = AutoModelForSeq2SeqLM.from_pretrained(
    checkpoint,
    quantization_config=quantization_config
)
```

Inference

At inference time, it is recommended to use the [generate function](#). This method takes care of encoding the input and auto-regressively generates the decoder output. Check out [this blog post](#) to know all the details about generating text with Transformers. There's also [this blog post](#) which explains how generation works in general in encoder-decoder models.

Let us first load the default inference parameters of NLLB.

```
In [ ]: from transformers import GenerationConfig

generation_config = GenerationConfig.from_pretrained(
    checkpoint,
)

print(generation_config)
```

We prepare the dataset in batches to be translated:

```
In [ ]: test_batch_size = 32
batch_tokenized_test = tokenized_datasets['train'].batch(test_batch_size)
```

Processing in batches to add padding and convert to tensors, then perform inference with `num_beams = 1` and `do_sample = False`, that is, greedy search.

```
In [ ]: number_of_batches = len(batch_tokenized_test["hypothesis"])
output_sequences = []
for i in range(number_of_batches):
    inputs = tokenizer(
        batch_tokenized_test["hypothesis"][i],
        max_length=max_tok_length,
        truncation=False,
        return_tensors="pt",
        padding=True,
    )
    output_batch = model.generate(
        generation_config=generation_config,
        input_ids=inputs["input_ids"].cuda(),
        attention_mask=inputs["attention_mask"].cuda(),
        forced_bos_token_id=tokenizer.convert_tokens_to_ids(tgt_code),
        max_length = max_tok_length,
        num_beams=1,
        do_sample=False,
    )
    output_sequences.extend(output_batch.cpu())
```

We can recover the decoded predictions and references by applying `batch_decode` of the tokenizer

```
In [11]: decoded_preds = tokenizer.batch_decode(output_sequences, skip_special_tokens=True)
```

```
In [12]: references = tokenizer.batch_decode(tokenized_datasets["train"]["labels"], skip_special_tokens=True)
```

Let's take a closer look at some decoded predictions and references:

```
In [ ]: decoded_preds[:1]
```

```
In [ ]: references[:2]
```

Predictions and references are normalized using the Whisper basic text standardisation/normalization module

```
In [15]: from whisper.normalizers.basic import BasicTextNormalizer

normalizer = BasicTextNormalizer()

decoded_preds_clean = [normalizer(text) for text in decoded_preds]
references_clean = [normalizer(text) for text in references]
```

For evaluation, we use the [Evaluate library](#) which includes the definition of generic and task-specific metrics. In our case, we use the [BLEU metric](#), or to be more precise, [sacreBLEU](#).

```
In [16]: from evaluate import load

metric = load("sacrebleu")
```

```
In [17]: result = metric.compute(predictions=decoded_preds_clean, references=references_clean)
print(f'BLEU score: {result["score"]:.1f}')
```

BLEU score: 9.5

Compute COMET figures using the [Evaluate library](#) which includes the definition of generic and task-specific metrics.

```
In [ ]: from evaluate import load
comet_metric = load('comet')
```

```
In [ ]: comet_score = comet_metric.compute(predictions=decoded_preds_clean, references=references_clean, sources=raw_dataset)
```

```
In [20]: print(f"COMET: {comet_score['mean_score'] * 100:.2f} %")
```

COMET: 57.91 %

Exercise

Perform a similar experiment using the Covost2 source-english setup previously used in L4.1. Evaluate the performance using the best whisper models obtained in L4.1.